
Software Requirements Specification

for

Carpooling System and Ride Allocation

Version 1.0 approved

Prepared by KISHORKUMAR V

15-10-2026

Table of Contents

Table of Contents	i
Revision History	ii
1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions	2
1.4 Product Scope	2
1.5 References	3
2. Overall Description	3
2.1 Product Perspective	3
2.2 Product Functions	4
2.3 User Classes and Characteristics	4
2.4 Operating Environment	10
2.5 Design and Implementation Constraints	10
2.6 User Documentation	11
2.7 Assumptions and Dependencies	11
3. External Interface Requirements	11
3.1 User Interfaces	11
3.2 Hardware Interfaces	12
3.3 Software Interfaces	12
3.4 Communications Interfaces	12
4. System Features	13
4.1 User Registration, Authentication and Profile Management	13
4.2 Ride Posting and Ride Management	13
4.3 Ride Search, Matching and Allocation	14
4.4 Booking and Ride Request Management	14
4.5 Notifications and Ride Updates	15
4.6 Feedback, Rating and Ride History	15
4.7 Administration, Complaints, Reports and Maintenance	16
5. Other Nonfunctional Requirements	16
5.1 Performance Requirements	16
5.2 Safety Requirements	16
5.3 Security Requirements	17
5.4 Software Quality Attributes	17
5.5 Business Rules	17
6. Other Requirements	18
Appendix A: Glossary	19
Appendix B: Analysis Models	20
B.1 UML Activity Diagrams	20
B.2 UML Sequence Diagrams	22
B.3 UML State Chart Diagram	24
B.4 UML Component Diagram	25
B.5 UML Deployment Diagram	26
Appendix C: To Be Determined List	26
Appendix D: Source Code	27
Appendix E: Demonstration	45
Appendix F: E-Resource Link	49

Revision History

Name	Date	Reason For Changes	Version
KISHORKUMAR V	September 2026	Initial completed SRS for Carpooling System and Ride Allocation	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Carpooling System and Ride Allocation application, Version 1.0.

The system is a web-based platform that connects drivers and passengers traveling along similar routes, enabling ride sharing and optimized ride allocation.

This document defines the complete scope of the system, including:

- User registration and authentication
- Ride posting
- Ride searching
- Ride matching and allocation
- Booking management
- Notifications
- Feedback and rating mechanisms
- Administrative functions

It serves as the primary reference for the design, development, testing, implementation, and maintenance of the system.

The primary objective of the platform is to:

- Reduce transportation costs
- Improve vehicle utilization
- Minimize traffic congestion
- Contribute to environmental sustainability through shared mobility

1.2 Document Conventions

This document follows the IEEE-recommended structure for Software Requirements Specifications. The following conventions apply:

- (1) Section headings follow a hierarchical decimal numbering system.
- (2) Each functional requirement is assigned a unique identifier in the format FR-N, non-functional requirements are prefixed with NFR-N, and business rules are prefixed with BR-N.
- (3) The terms "shall", "must", and "will" indicate mandatory requirements, "should" indicates recommended requirements, and "may" indicates optional features.
- (4) Important terms, module names, and system components are highlighted when first introduced.
- (5) User interface elements such as buttons, menus, and forms are represented in italics where applicable.

- (6) Requirement priorities are classified as High (essential for the core functionality of the system), Medium (important but not critical for the initial release), and Low (optional enhancements for future versions).
- (7) Each requirement is assigned its own priority level rather than inheriting the priority of a higher-level requirement.
- (8) User roles such as Passenger, Driver, Administrator, and Guest User are explicitly identified throughout the document.
- (9) Diagrams, tables, and figures are numbered and titled for easy reference.

1.3 Intended Audience and Reading Suggestions

This SRS document is intended for the following audiences:

- (1) **Project Managers**
 - Responsible for project planning, resource allocation, and progress tracking.
 - Should focus on Sections 1, 2, and 5.
- (2) **Software Developers**
 - Responsible for designing and implementing the system modules.
 - Should read all sections thoroughly.
- (3) **System Analysts and Designers**
 - Responsible for creating the system architecture and design models.
 - Should focus on Sections 2, 3, and 4 and Appendix B.
- (4) **Testers and Quality Assurance Engineers**
 - Responsible for verifying that the system satisfies all specified requirements.
 - Should focus on Sections 3, 4, and 5.
- (5) **Database Administrators**
 - Responsible for data storage, database structure, and data security.
 - Should focus on the data requirements in Sections 3, 4, and 6.
- (6) **End Users (Drivers and Passengers)**
 - Users who post, search, and book rides.
 - Should focus on Sections 1, 2, and 4.
- (7) **Documentation and Maintenance Teams**
 - Responsible for user documentation and future system updates.
 - Should review the entire document.

It is recommended that all readers begin with Section 1 and Section 2 to gain a comprehensive understanding of the product before proceeding to the sections relevant to their role.

1.4 Product Scope

The Carpooling System and Ride Allocation is a web-based application designed to facilitate ride sharing by connecting drivers and passengers traveling in the same or similar directions.

The system enables users to offer rides, search for available rides, request bookings, and allocate seats based on route compatibility, travel schedules, and seat availability.

The primary purpose of the system is to provide an affordable, convenient, and environmentally sustainable transportation solution by encouraging multiple passengers to share a single vehicle, thereby reducing transportation costs, traffic congestion, fuel consumption, and carbon emissions.

Key objectives include:

- Providing a secure platform for ride sharing between registered users
- Optimizing ride allocation through intelligent matching of drivers and passengers
- Improving vehicle occupancy and reducing the number of vehicles on the road
- Offering a user-friendly interface for ride management and booking
- Enhancing user trust through ratings, feedback, and profile verification
- Supporting efficient communication and notifications between drivers and passengers

The system aligns with smart transportation, cost reduction, and sustainability goals, and is designed to be a reliable and scalable ride-sharing platform that benefits both individual users and society.

1.5 References

The following references were consulted during the preparation of this SRS:

- [1] IEEE Std 830-1998, "IEEE Recommended Practice for Software Requirements Specifications," IEEE, 1998.
- [2] ISO/IEC/IEEE 29148:2018, "Systems and Software Engineering – Life Cycle Processes – Requirements Engineering," 2018.
- [3] Unified Modeling Language (UML), Version 2.5.1, Object Management Group, 2017.
<https://www.omg.org/spec/UML/2.5.1>
- [4] MySQL 8.0 Reference Manual, Oracle Corporation. <https://dev.mysql.com/doc/>
- [5] PostgreSQL Documentation, The PostgreSQL Global Development Group.
<https://www.postgresql.org/docs/>
- [6] React Documentation, Meta Open Source. <https://react.dev/>
- [7] Node.js Documentation, OpenJS Foundation. <https://nodejs.org/docs/latest/api/>
- [8] Express Documentation, OpenJS Foundation. <https://expressjs.com/>

2. Overall Description

2.1 Product Perspective

The Carpooling System and Ride Allocation is a new, self-contained web application developed to facilitate ride-sharing services between drivers and passengers.

It addresses common transportation challenges such as:

- Traffic congestion
- High travel costs
- Inefficient vehicle utilization
- Environmental pollution caused by excessive use of private vehicles

The application serves as a centralized platform where drivers can offer rides and passengers can search, request, and book available rides according to their travel requirements.

Ride allocation mechanisms match passengers with suitable drivers by considering route compatibility, travel schedules, and seat availability.

The product is not a replacement for any existing organizational system but is inspired by modern ride-sharing and carpooling platforms.

The system consists of the following major components:

- (1) **User Management Module** – handles user registration, authentication, profile management, and account security.
- (2) **Ride Management Module** – enables drivers to create, modify, and manage ride offers.
- (3) **Ride Search and Allocation Module** – matches passengers with available rides and allocates seats efficiently.
- (4) **Booking Management Module** – processes ride requests, confirmations, cancellations, and booking records.
- (5) **Notification Module** – sends ride updates, booking confirmations, and reminders to users.
- (6) **Feedback and Rating Module** – allows users to provide ratings and reviews based on ride experiences.
- (7) **Database Management System** – stores user information, ride details, booking records, ratings, and system logs.

The system interacts with external services such as email or SMS notification services and may integrate with mapping and location services for route identification and navigation support.

2.2 Product Functions

The Carpooling System and Ride Allocation shall provide the following major functions:

- (1) **User Registration and Authentication** – allow users to create accounts, securely log in, and manage their profiles.
- (2) **Ride Posting** – enable drivers to create and publish ride offers by specifying source, destination, travel date, time, fare, and available seats.
- (3) **Ride Search** – allow passengers to search for available rides based on location, date, and time preferences and view matching ride options.
- (4) **Ride Allocation and Matching** – match passengers with suitable drivers based on route compatibility and seat availability to optimize vehicle utilization.
- (5) **Ride Booking Management** – allow passengers to request rides, enable drivers to approve or reject requests, and manage confirmations and cancellations.
- (6) **Notification Services** – send booking confirmations, ride updates, reminders, and cancellation notifications.
- (7) **Feedback and Rating System** – allow passengers and drivers to rate and review each other after completed trips.
- (8) **Administration and Monitoring** – enable administrators to manage users, rides, complaints, and system activities and to generate reports.

- (9) **Database Management** – store and manage user details, ride information, booking records, ratings, and transaction history securely.

2.3 User Classes and Characteristics

The system serves four user classes:

(1) **Passenger**

- Primary user of high importance who seeks transportation by sharing rides with drivers.
- Has basic computer and smartphone usage knowledge.
- Registers and logs in to the system.
- Searches available rides.
- Requests, cancels, and manages bookings.
- Views ride status and notifications.
- Submits ratings and reviews.

(2) **Driver**

- Primary user of high importance who offers available seats in a vehicle.
- Possesses a valid driving license and vehicle details.
- Maintains a driver profile.
- Posts, updates, and cancels rides.
- Accepts or rejects passenger requests.
- Manages seat availability.
- Views ride history.
- Provides ratings and reviews.

(3) **System Administrator**

- Highest-privilege user of medium importance who manages and monitors overall system operation.
- Has technical knowledge of system management.
- Manages user accounts and approves or suspends users.
- Monitors ride and booking activities.
- Handles complaints and disputes.
- Generates reports.
- Manages system settings and maintenance tasks.

(4) **Guest User**

- Optional user of low importance who accesses the platform without an account.
- Can view general information.
- Can access the registration and login pages.
- Cannot post rides or make bookings.

The use case diagrams and corresponding use case descriptions for each user class of the Carpooling System and Ride Allocation are given below.

2.3.1 Passenger

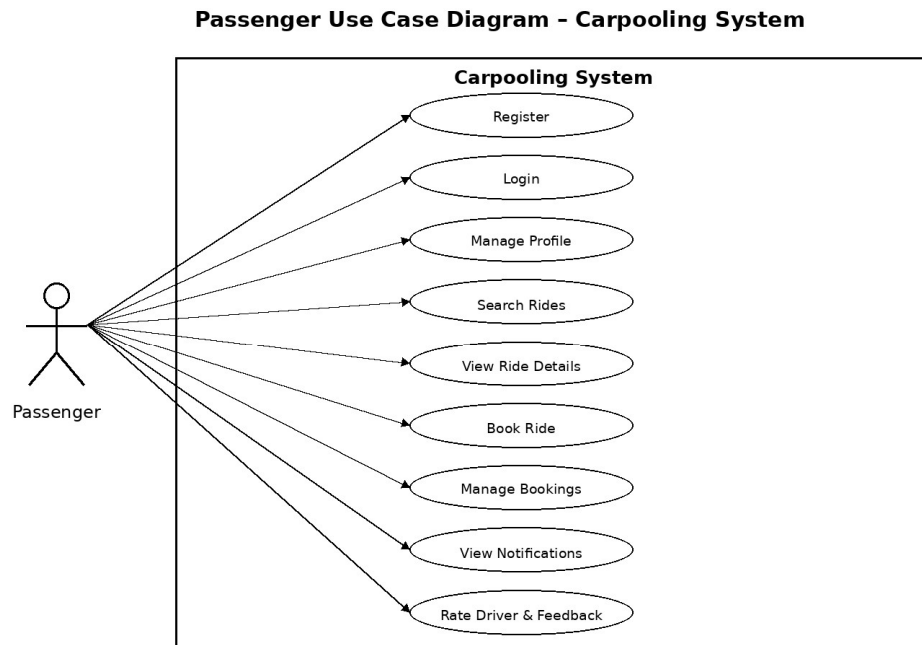


Figure 1: Passenger Use Case Diagram

Use Case	Passenger Operations
Primary Actor	Passenger
Secondary Actors	Authentication Service, Ride Allocation System, Booking Management System, Notification Service, Database
Precondition	Registered and verified account; passenger is logged in.
Postcondition	Selected operation is completed and relevant booking information is updated.
Trigger	Passenger selects a required operation.
Main Access Scenario	1. Passenger logs in. 2. Passenger searches and views rides. 3. Passenger selects a ride and books seats. 4. System confirms booking and updates availability. 5. Passenger views history/notifications and gives feedback.
Exception Scenario	• No matching ride is available. • Booking or feedback fails due to a system error.

2.3.2 Driver

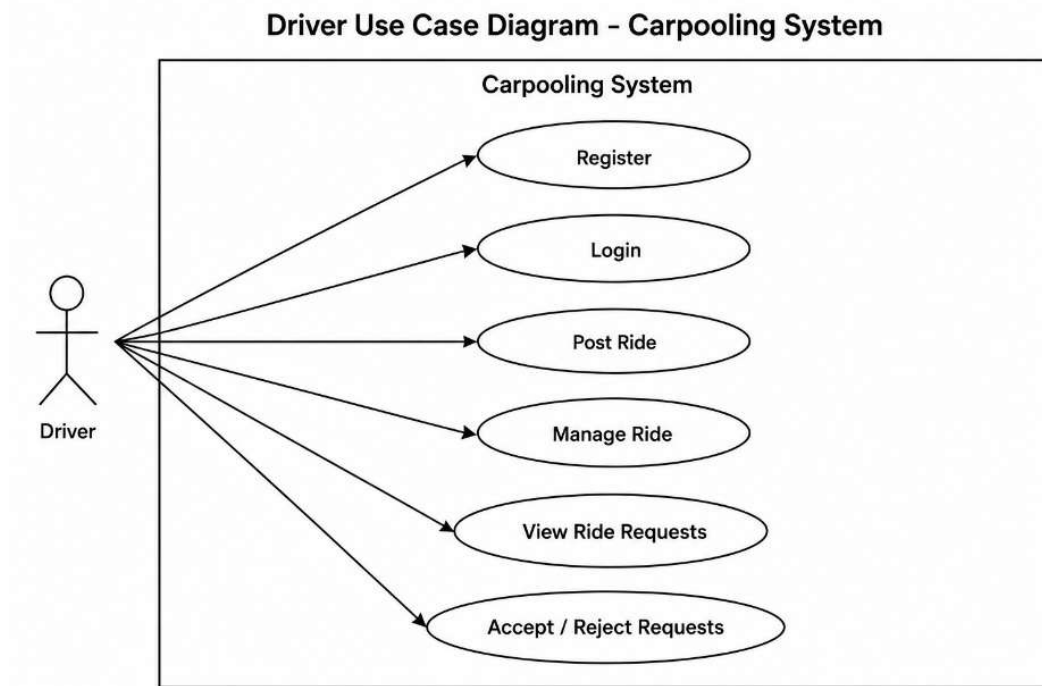


Figure 2: Driver Use Case Diagram

Use Case	Driver Operations
Primary Actor	Driver
Secondary Actors	Authentication Service, Ride Management System, Booking Management System, Notification Service, Database
Precondition	Registered and verified account; driver is logged in.
Postcondition	Ride and booking information is updated and confirmation is provided.
Trigger	Driver selects an operation such as posting a ride or managing requests.
Main Access Scenario	1. Driver logs in. 2. Driver posts or manages a ride. 3. Driver views and responds to passenger requests. 4. System updates booking status and notifies passengers. 5. Driver manages confirmed rides and provides feedback.
Exception Scenario	• Ride details are invalid or incomplete. • Request or booking update fails due to a system error.

2.3.3 Administrator

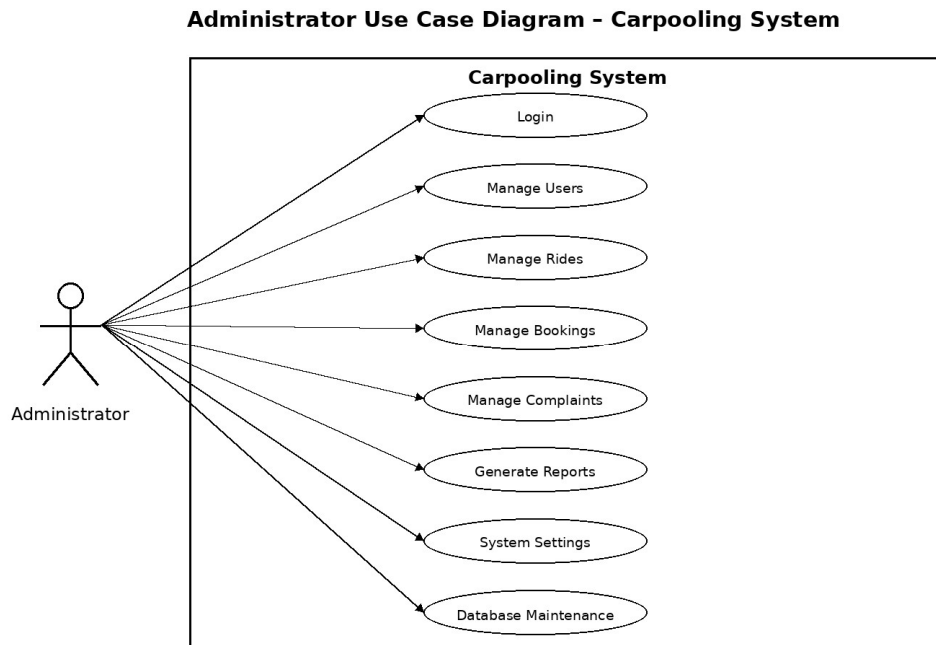


Figure 3: Administrator Use Case Diagram

Use Case	Administrator Operations
Primary Actor	Administrator
Secondary Actors	Authentication Service, User Management System, Ride Management System, Booking Management System, Complaint Management System, Report Generation System, Database
Precondition	Valid administrative credentials; administrator is authenticated.
Postcondition	Administrative changes are saved and system records are updated.
Trigger	Administrator selects an administrative operation.
Main Access Scenario	1. Administrator logs in. 2. Administrator manages users, rides, and bookings. 3. Administrator handles complaints and generates reports. 4. Administrator configures system settings. 5. System saves changes and records administrative activity.
Exception Scenario	• Operation is restricted due to insufficient permission. • Database or system error prevents completion.

2.3.4 Guest User

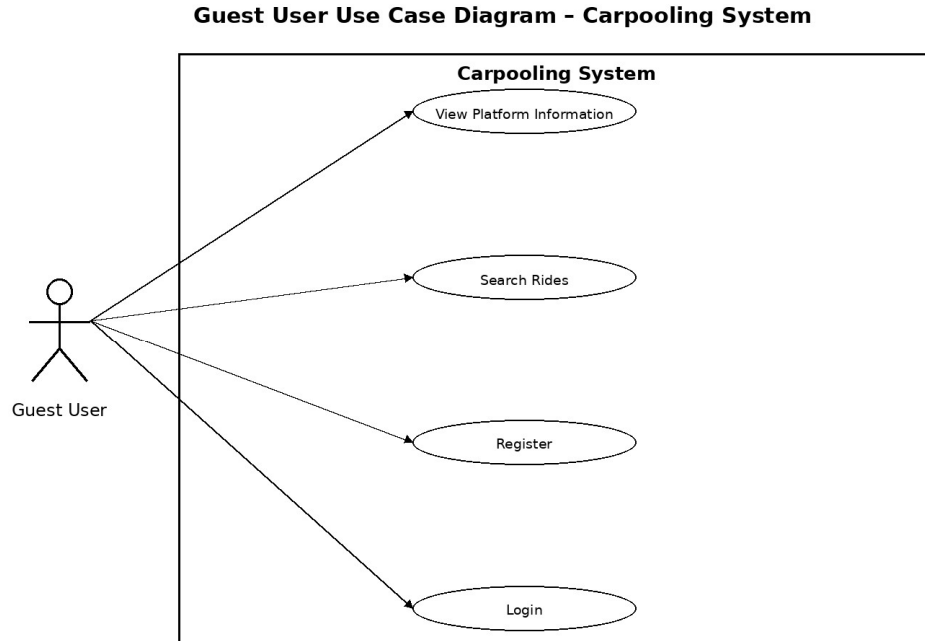


Figure 4: Guest User Use Case Diagram

Use Case	Guest User Operations
Primary Actor	Guest User
Secondary Actors	Authentication Service, Ride Search System, Registration Service, Database
Precondition	Guest can access the application without an account.
Postcondition	Requested public information is displayed or a new account is created.
Trigger	Guest accesses the application and selects a public operation.
Main Access Scenario	1. Guest views platform information. 2. Guest searches for available rides. 3. System displays matching ride summaries. 4. Guest registers if required. 5. System validates registration and allows login.
Exception Scenario	• No matching rides are available. • Registration or login fails due to invalid details or system error.

2.4 Operating Environment

The Carpooling System and Ride Allocation shall operate in the following environment:

- (1) **Client Side**
 - Desktop or laptop computer: Intel Core i3 or equivalent, 4 GB RAM or higher, and 500 MB free storage.
 - Smartphone or tablet: Android 10+ or iOS 14+ with a minimum of 2 GB RAM.
 - Browsers: latest version of Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari.
 - Operating systems: Windows 10/11, Ubuntu 20.04+, or macOS Monterey or later.
- (2) **Server Side**
 - Cloud or dedicated hosting environment.
 - Intel Core i5 or higher processor.
 - 8 GB RAM or higher.
 - 100 GB or more storage.
- (3) **Database** – MySQL 8.0+ or PostgreSQL 14+ for storing user, ride, booking, notification, and feedback data.
- (4) **Development Technologies**
 - Frontend: HTML5, CSS3, JavaScript, and React.js.
 - Backend: Node.js and Express.js.
- (5) **Network**
 - A reliable internet connection supporting multiple concurrent users.
 - HTTPS used to secure all client-server communication.
- (6) **External Services**
 - Email services for verification and notifications.
 - SMS services for ride updates.
 - GPS and map services for route identification.
 - Payment gateway services if online payment is implemented in future versions.

2.5 Design and Implementation Constraints

The following constraints shall apply to the development of the system:

- (1) The implementation shall use the technologies and database selected for the project and shall follow a standard web application architecture.
- (2) The system shall require secure authentication and role-based access control for Passenger, Driver, Administrator, and Guest users.
- (3) Ride allocation depends on route, date/time, and available-seat data.
- (4) The application depends on reliable network connectivity for server and database access and may depend on external email/SMS and mapping services.
- (5) Any future payment integration shall use an approved payment provider rather than storing raw payment credentials.
- (6) The design shall follow applicable institutional policies, secure coding practices, and database integrity constraints.

2.6 User Documentation

The following user documentation shall be delivered along with the system:

- (1) **User Guide** – a short guide covering:
 - Registration, login, and profile management
 - Ride search and ride posting
 - Booking and request management, and cancellations
 - Notifications, ratings, and feedback
- (2) **Administrator Guide** – instructions for:
 - User moderation
 - Ride and booking management
 - Complaint handling and reporting
 - System configuration and database maintenance
- (3) **Online Help** – contextual guidance for important screens and validation errors.

2.7 Assumptions and Dependencies

Assumptions

- (1) It is assumed that users provide accurate profile, vehicle, route, and booking information.
- (2) It is assumed that drivers possess valid driving licenses and vehicle documentation where required.
- (3) It is assumed that users have access to a supported web browser and a reliable internet connection.
- (4) It is assumed that the database service remains available during the service period.
- (5) It is assumed that external notification or mapping services, when enabled, expose stable interfaces.

Dependencies

- (1) The system depends on the selected database platform (MySQL 8.0+ or PostgreSQL 14+).
- (2) The system depends on the selected web server and runtime environment (Node.js and Express.js).
- (3) The application depends on supported modern web browsers for client access.
- (4) The system depends on any configured email, SMS, or map service when those features are enabled.

3. External Interface Requirements

3.1 User Interfaces

- The user interface shall provide separate role-appropriate dashboards for Passenger, Driver, Administrator, and limited Guest access.
- Common controls include navigation, search, forms, confirmation dialogs, validation messages, notifications, profile/account controls, and logout.

- Passenger screens shall support ride search, ride details, booking requests, booking history, cancellations, notifications, and feedback.
- Driver screens shall support ride creation, update, and deletion, seat availability, passenger requests, booking management, ride history, earnings/history, and feedback.
- Administrator screens shall support user, ride, booking, complaint, report, settings, and database-management functions.
- Error messages shall identify the problem and provide a clear corrective action.

3.2 Hardware Interfaces

- The system shall support desktop/laptop computers, smartphones, and tablets through a modern web browser.
- Client devices shall communicate with the application server through HTTPS.
- No specialized hardware is required for normal operation.
- Optional location features may use device or browser location capabilities when permission is granted.
- The server hardware shall provide sufficient CPU, memory, storage, and network capacity for concurrent users and database operations.

3.3 Software Interfaces

- The application shall use a relational database such as MySQL 8.0+ or PostgreSQL 14+ for storing users, rides, bookings, notifications, feedback, complaints, and audit data.
- The web client shall communicate with the application backend through HTTP/HTTPS APIs.
- Optional integrations include email/SMS services for verification and notifications and map/GPS services for route identification.
- The implementation shall validate all external inputs and handle unavailable external services gracefully.

3.4 Communications Interfaces

- All browser-to-server communication shall use HTTPS.
- Application APIs shall use standard HTTP methods and structured request and response data such as JSON.
- Authentication and session data shall be protected in transit.
- Email and SMS notifications shall contain only the information necessary for the event.
- Network failures, timeouts, and unavailable third-party services shall produce user-friendly error messages and shall not corrupt booking or ride records.

4. System Features

4.1 User Registration, Authentication and Profile Management

4.1.1 Description and Priority

Priority: High

Provides account creation, verification, secure login and logout, profile updates, and role-aware access for passengers and drivers.

4.1.2 Stimulus/Response Sequences

- User selects Register or Login and enters the required credentials.
- System validates the input and checks the account status.
- For registration, the system creates the account and performs email/phone verification when configured.
- For successful login, the system creates an authenticated session and opens the appropriate dashboard.
- Invalid credentials or incomplete input are rejected with a clear error message.

4.1.3 Functional Requirements

- FR-1:** The system shall allow a guest to register with name, email, phone, password, and required role information.
- FR-2:** The system shall validate unique email/phone values and password requirements before creating an account.
- FR-3:** The system shall authenticate registered users and provide role-based access.
- FR-4:** The system shall allow authenticated users to view and update permitted profile information.
- FR-5:** The system shall prevent suspended or disabled accounts from accessing protected functions.

4.2 Ride Posting and Ride Management

4.2.1 Description and Priority

Priority: High

Enables drivers to create and manage ride offers including route, schedule, fare, and seat availability.

4.2.2 Stimulus/Response Sequences

- Driver selects Post Ride and enters source, destination, date, time, fare, and available seats.
- System validates the ride details and creates the ride offer.
- Driver may update ride details or available seats before the ride is completed.
- Driver may delete or cancel a ride subject to booking constraints.
- The system notifies affected passengers when a confirmed ride is changed or cancelled.

4.2.3 Functional Requirements

- FR-6:** The system shall allow an authenticated driver to create a ride offer with mandatory route, date/time, fare, and seat details.
- FR-7:** The system shall reject invalid dates, times, negative fares, and invalid seat counts.
- FR-8:** The system shall allow the driver to update or delete eligible ride offers.
- FR-9:** The system shall prevent available seats from becoming less than already confirmed bookings.
- FR-10:** The system shall record ride status and relevant changes.

4.3 Ride Search, Matching and Allocation

4.3.1 Description and Priority

Priority: High

Allows passengers to search for rides and supports allocation based on route compatibility, schedule, and available seats.

4.3.2 Stimulus/Response Sequences

- Passenger enters source, destination, date, and time criteria.
- System retrieves active rides satisfying the search criteria.
- System displays matching ride summaries including route, time, fare, and available seats.
- Passenger selects a suitable ride and requests a booking.
- The system checks current seat availability before creating the request.

4.3.3 Functional Requirements

- FR-11:** The system shall allow passengers to search rides by source, destination, date, and time.
- FR-12:** The system shall display only rides that are active and have relevant seat availability.
- FR-13:** The system shall support route/time compatibility checks for ride allocation.
- FR-14:** The system shall prevent booking requests when no eligible ride or seat is available.
- FR-15:** Search results shall show sufficient ride information for the passenger to make a booking decision.

4.4 Booking and Ride Request Management

4.4.1 Description and Priority

Priority: High

Handles passenger requests, driver approval/rejection, confirmation, cancellation, and booking history.

4.4.2 Stimulus/Response Sequences

- Passenger submits a booking request for a selected ride.
- System records the request as pending and notifies the driver.
- Driver reviews the passenger request and accepts or rejects it.
- If accepted, the system confirms the booking and updates seat availability.

- If rejected or cancelled, the system updates the booking status and notifies the relevant user.

4.4.3 Functional Requirements

- FR-16:** The system shall create a unique booking record for each valid request.
- FR-17:** The system shall support Pending, Confirmed, Cancelled, and Completed booking states.
- FR-18:** The system shall allow drivers to accept or reject eligible requests.
- FR-19:** The system shall update available seats atomically when a booking is confirmed or cancelled.
- FR-20:** The system shall prevent duplicate or conflicting confirmed bookings for the same passenger and ride.

4.5 Notifications and Ride Updates

4.5.1 Description and Priority

Priority: High

Keeps users informed about registration, booking, ride changes, cancellations, reminders, and other relevant events.

4.5.2 Stimulus/Response Sequences

- A system event such as a booking request, approval, cancellation, or ride change occurs.
- System creates an in-app notification and, when configured, sends an email/SMS notification.
- User opens the notification list and views unread/read status.
- Notification failures are logged without corrupting the underlying transaction.

4.5.3 Functional Requirements

- FR-21:** The system shall generate notifications for important booking and ride events.
- FR-22:** The system shall maintain notification type, message, timestamp, recipient, and read status.
- FR-23:** Users shall be able to view and mark notifications as read.
- FR-24:** External notification failures shall not roll back a successfully completed core booking transaction.

4.6 Feedback, Rating and Ride History

4.6.1 Description and Priority

Priority: Medium

Allows passengers and drivers to review completed rides, rate one another, and view historical activity.

4.6.2 Stimulus/Response Sequences

- A ride reaches completed status.
- Eligible users open the completed ride and submit a rating and optional comments.
- System validates and stores the feedback.
- Users can view permitted ride history, ratings, and related summaries.

4.6.3 Functional Requirements

- FR-25:** The system shall allow eligible users to rate and review one another only for completed rides.
- FR-26:** Ratings shall be validated against the configured rating scale.
- FR-27:** The system shall store feedback with the related ride, users, rating, comments, and timestamp.
- FR-28:** The system shall provide passengers and drivers with their relevant ride history.

4.7 Administration, Complaints, Reports and Maintenance

4.7.1 Description and Priority

Priority: High

Provides administrators with controlled tools for user moderation, ride/booking management, complaints, reporting, settings, and database maintenance.

4.7.2 Stimulus/Response Sequences

- Administrator logs in and is authenticated with administrator privileges.
- Administrator selects a management function.
- System verifies authorization and displays the relevant records.
- Administrator performs an approved action such as approve/suspend user, manage rides/bookings, resolve complaints, or generate reports.
- System records the action and updates the affected data.

4.7.3 Functional Requirements

- FR-29:** The system shall restrict administrator functions to authorized administrator accounts.
- FR-30:** Administrators shall be able to view, approve, suspend, edit, or delete user accounts according to policy.
- FR-31:** Administrators shall be able to monitor and manage rides and bookings.
- FR-32:** Administrators shall be able to view, assign, resolve, and close complaints or disputes.
- FR-33:** The system shall generate user, ride, booking, earnings/activity, and operational reports.
- FR-34:** The system shall support controlled database backup/restore and maintenance operations.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- NFR-1:** Normal user actions such as login, ride search, and dashboard loading should normally receive a response within 3 seconds under expected operating conditions.
- NFR-2:** The system shall support multiple concurrent users without data corruption or conflicting booking updates.
- NFR-3:** Search operations shall use indexed fields for commonly queried route, date, and time data where supported by the database.

NFR-4: Booking confirmation shall update seat availability consistently so that the same seat cannot be allocated twice.

5.2 Safety Requirements

NFR-5: The system shall prevent accidental loss of confirmed booking and ride data through transactional database updates.

NFR-6: Deleting or cancelling a ride with confirmed bookings shall follow the configured business rules and shall notify affected users.

NFR-7: Database backup procedures shall be available to administrators.

NFR-8: The system shall display clear warnings before destructive administrative operations.

5.3 Security Requirements

NFR-9: Passwords shall never be stored in plain text and shall be protected using a secure password-hashing mechanism.

NFR-10: Role-based authorization shall restrict Passenger, Driver, Administrator, and Guest functions.

NFR-11: All protected communication shall use HTTPS.

NFR-12: The system shall validate and sanitize user input to reduce injection and other common web attacks.

NFR-13: Sessions shall expire or be invalidated according to the application's authentication policy.

NFR-14: Sensitive personal information shall be accessible only to authorized users and administrators with a legitimate system purpose.

5.4 Software Quality Attributes

- **Usability (NFR-15):** Core passenger and driver tasks shall be understandable with clear labels, validation, and navigation.
- **Reliability (NFR-16):** Booking and seat-allocation operations shall preserve database consistency even when a request fails.
- **Maintainability (NFR-17):** The system shall use modular components and documented interfaces so individual services can be changed without rewriting unrelated modules.
- **Portability (NFR-18):** The web application shall operate on supported modern browsers across desktop and mobile devices.
- **Testability (NFR-19):** Functional requirements shall be independently testable through defined inputs and expected outputs.
- **Availability (NFR-20):** The deployed application should remain available during the expected service period except for planned maintenance.

5.5 Business Rules

BR-1: Only authenticated passengers may request or book rides.

BR-2: Only authenticated drivers may create and manage ride offers.

BR-3: A driver cannot confirm more passengers than the available seat capacity.

- BR-4:** A passenger cannot hold conflicting confirmed bookings for the same travel period where the system's scheduling rules prohibit them.
- BR-5:** Ratings and feedback may be submitted only for completed rides.
- BR-6:** Administrators have privileged access to moderation, reporting, and maintenance functions.
- BR-7:** Guest users may browse permitted public information but cannot perform protected ride or booking operations.
- BR-8:** Cancellation and refund/payment rules, if payment is introduced later, shall follow the configured business policy.

6. Other Requirements

Database requirements:

- The system shall maintain consistent records for users, rides, bookings, notifications, feedback, complaints, and audit information.
- Data relationships and referential integrity shall be enforced where applicable.
- The design should support future localization, accessibility improvements, mapping integration, and optional online payment integration without changing the core ride-allocation model.
- The system shall retain only the data necessary for its defined business functions.
- The system shall support controlled data backup and recovery.

Appendix A: Glossary

Term	Definition
Carpooling	Shared travel in which multiple passengers use seats in a driver's vehicle.
Ride	A driver's offered trip with route, schedule, fare, and seat information.
Ride Offer	A published ride available for passenger requests.
Booking	A passenger's recorded request or confirmation for a seat on a ride.
Ride Allocation	Matching and assigning passengers to suitable rides based on route, time, and seat availability.
Driver	A registered user who offers a ride.
Passenger	A registered user who requests or books a ride.
Guest User	A visitor with limited access before registration or login.
Administrator	A privileged user responsible for system management and monitoring.
Notification	A system-generated message informing a user about an event or update.
Feedback	A rating and review submitted after a completed ride.
Role-Based Access Control (RBAC)	A security mechanism that restricts operations according to the user's assigned role.
HTTPS	The secure HTTP protocol used for encrypted client-server communication.

Appendix B: Analysis Models

The following analysis models support the requirements and design of the Carpooling System and Ride Allocation application.

B.1 UML Activity Diagrams

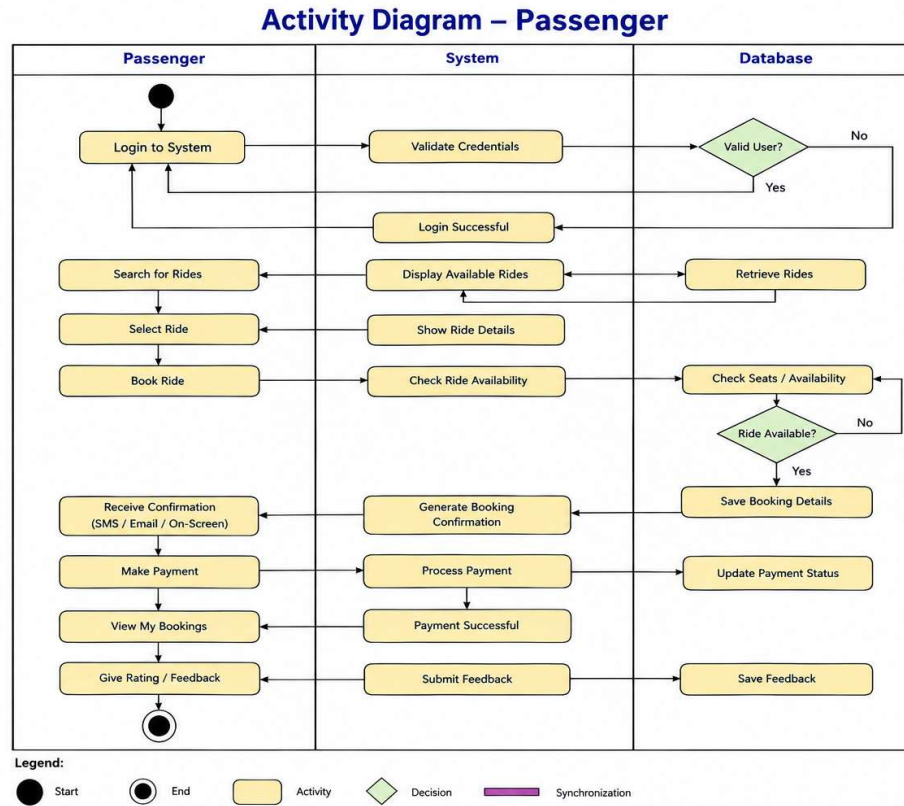


Figure 5: UML Activity Diagram – Passenger

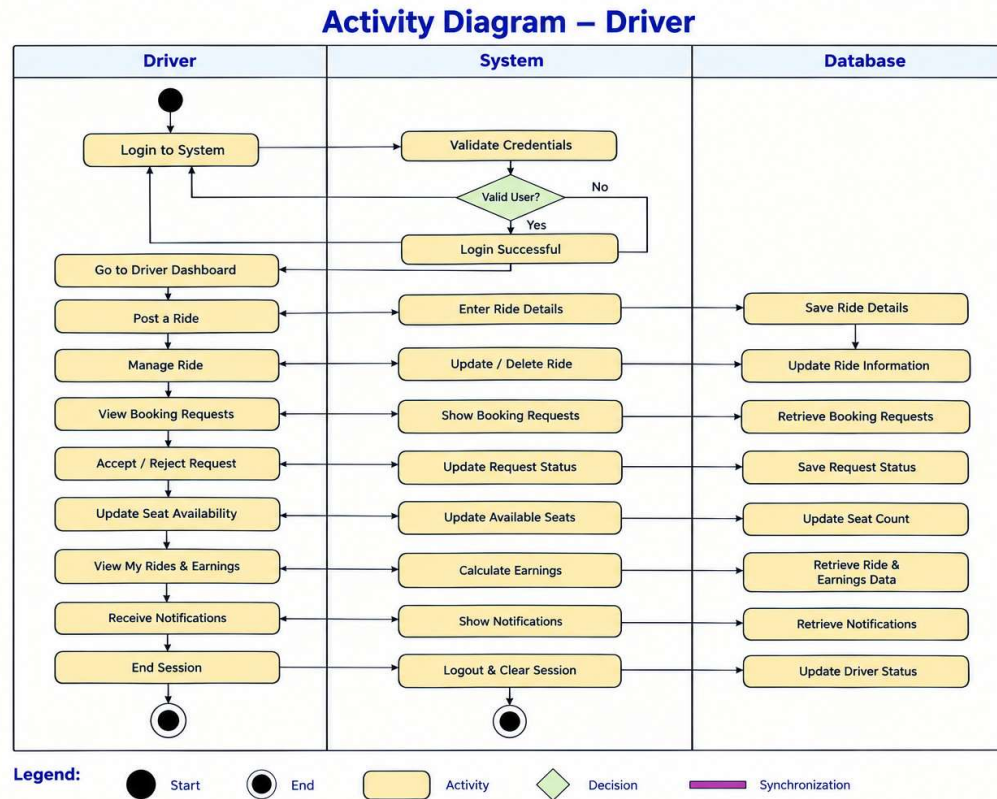


Figure 6: UML Activity Diagram – Driver

B.2 UML Sequence Diagrams

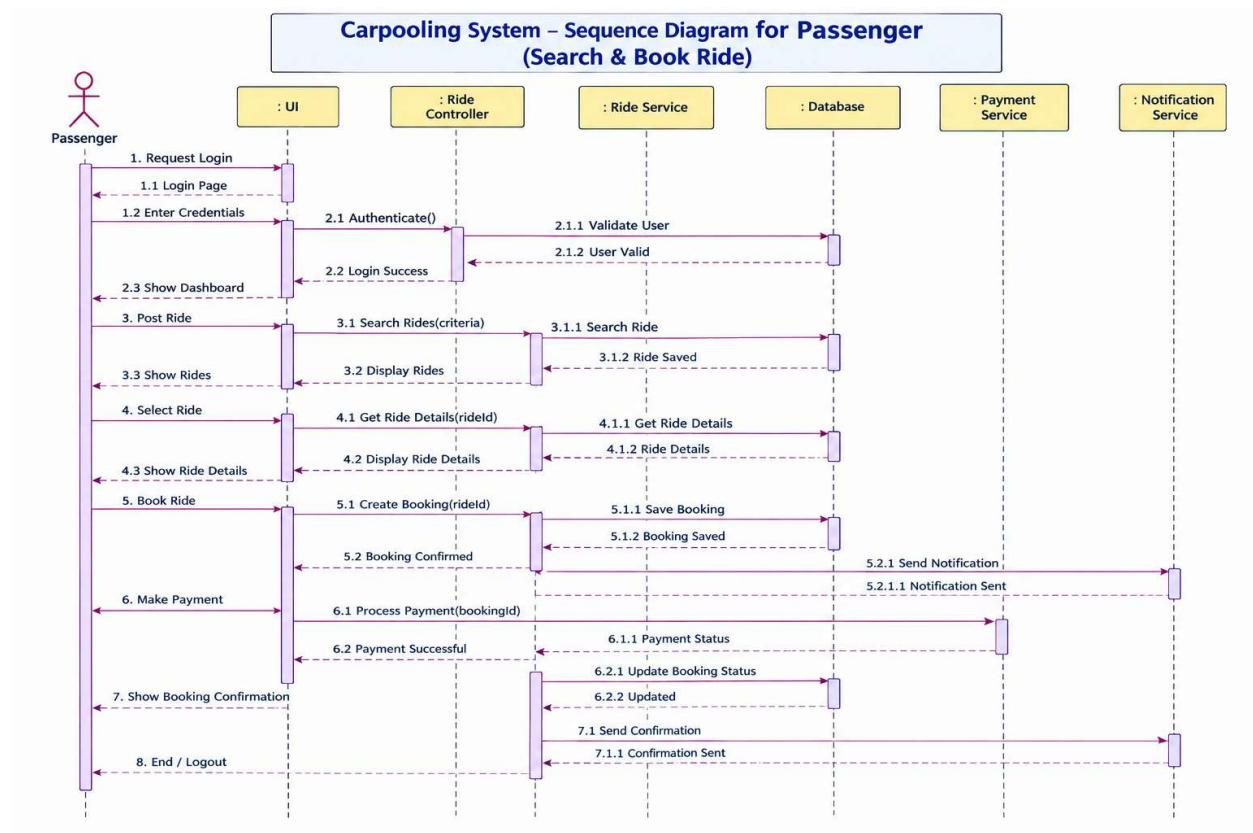


Figure 7: UML Sequence Diagram – Passenger (Search & Book Ride)

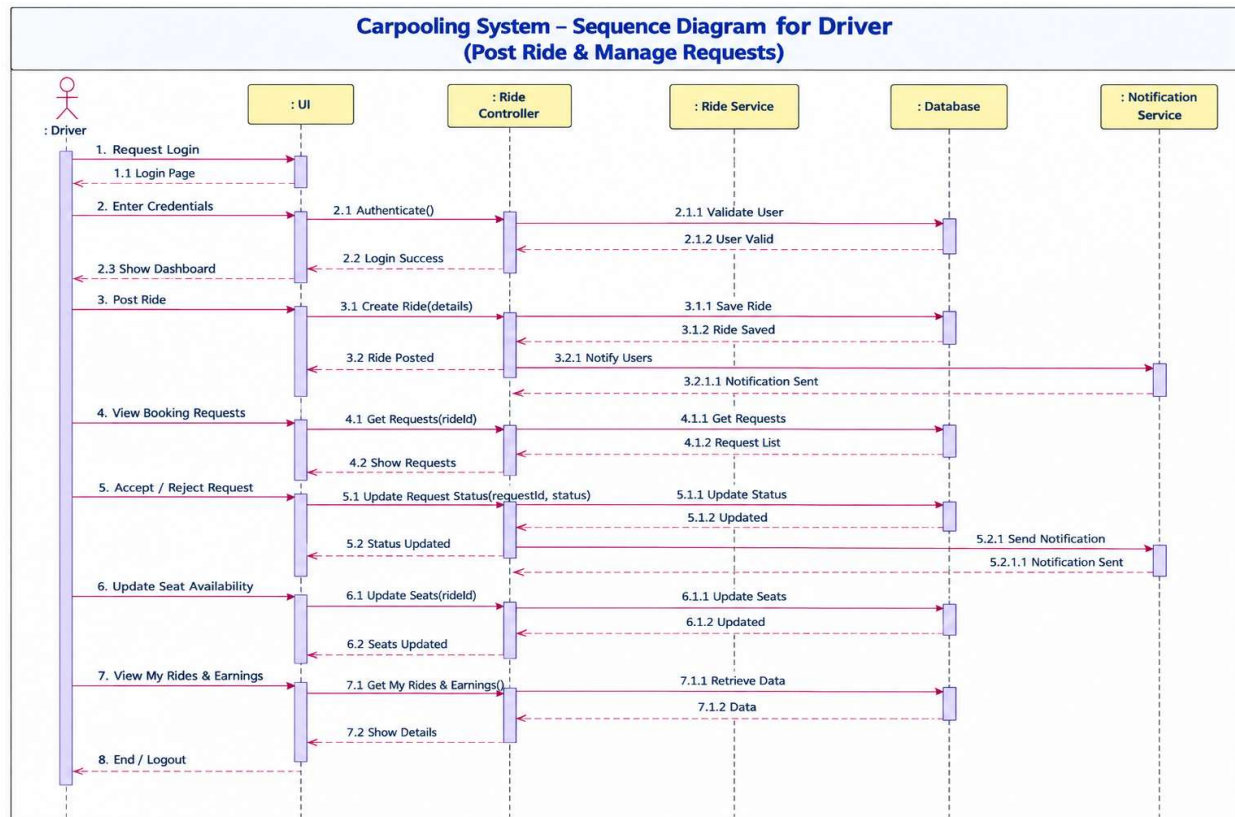


Figure 8: UML Sequence Diagram – Driver (Post Ride & Manage Requests)

B.3 UML State Chart Diagram

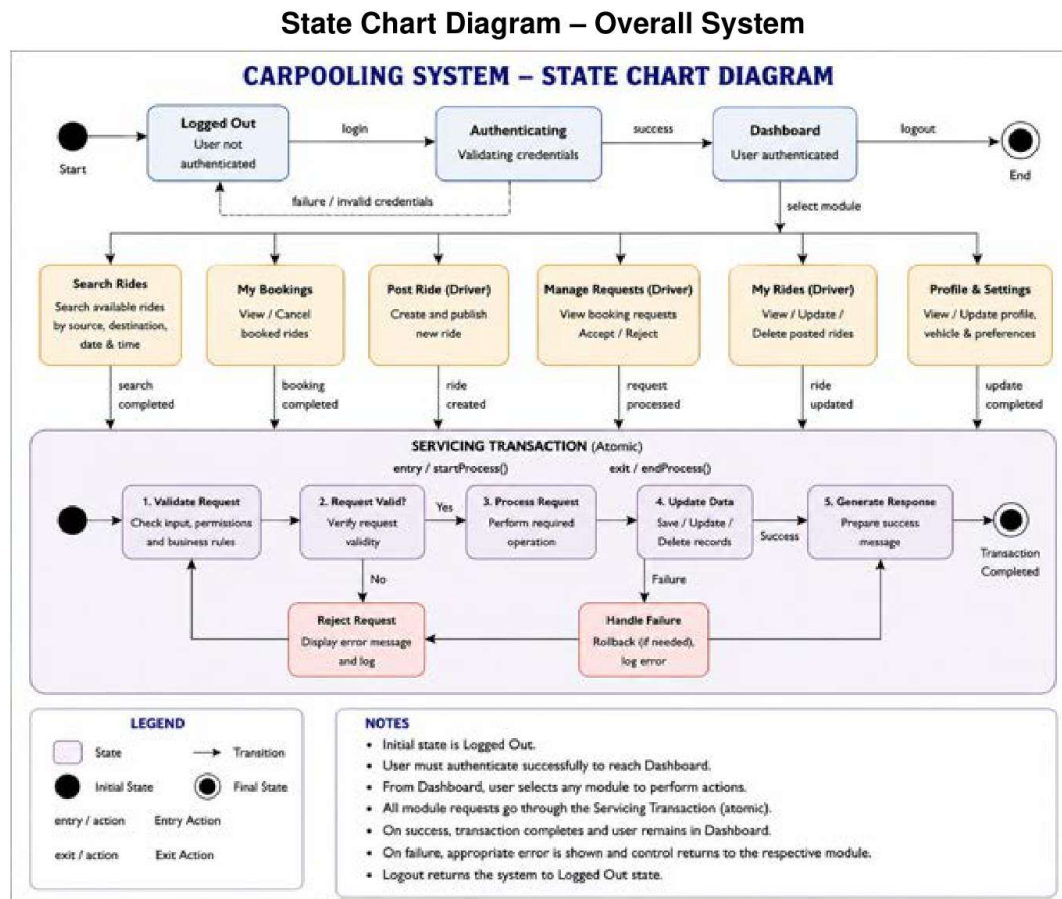


Figure 9: UML State Chart Diagram – Carpooling System

B.4 UML Component Diagram

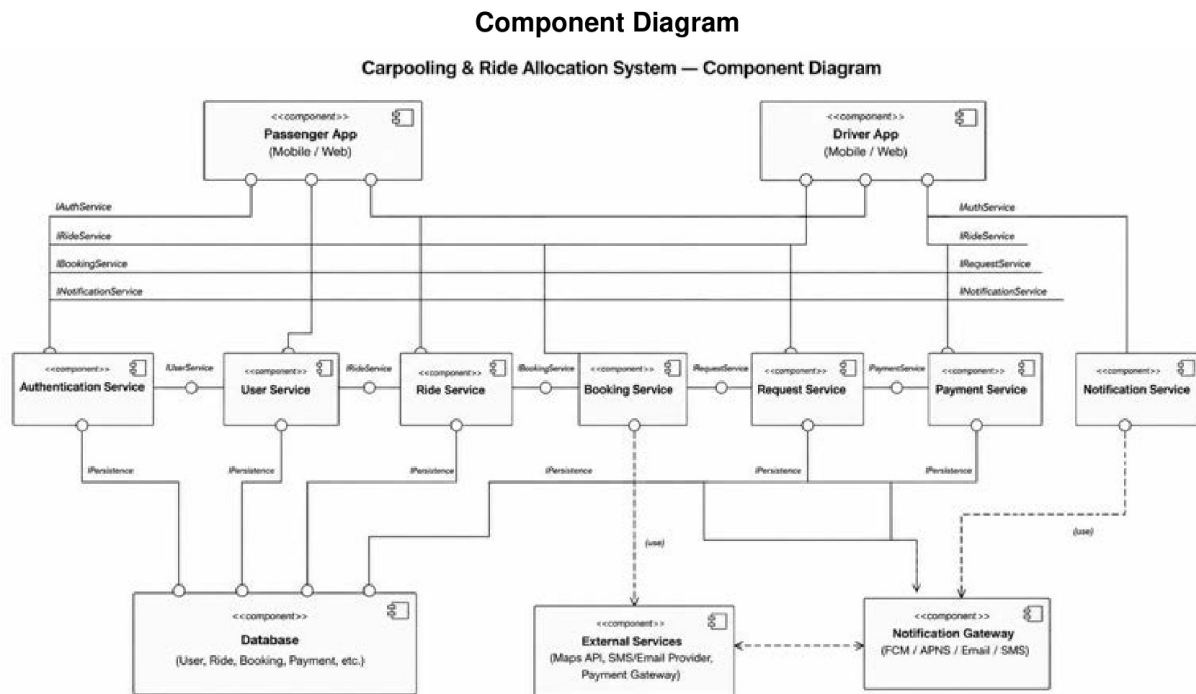


Figure 10: UML Component Diagram – Carpooling System

B.5 UML Deployment Diagram

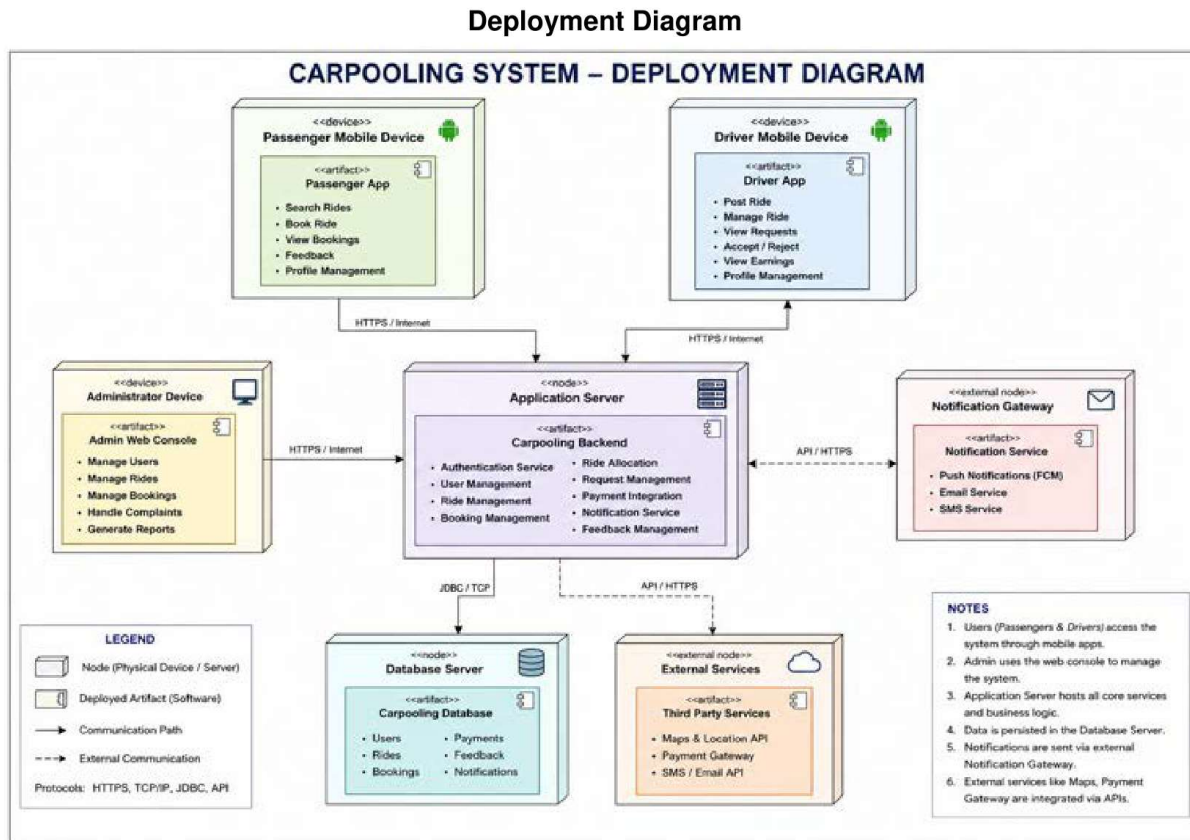


Figure 11: UML Deployment Diagram – Carpooling System

The analysis models are consistent with the Version 1.0 role model of Guest User, Passenger, Driver, and Administrator. Online payment remains a future/optional integration and is not a mandatory core Version 1.0 function.

Appendix C: To Be Determined List

- TBD-1:** Online payment integration and payment provider selection (future enhancement; not part of Version 1.0).
- TBD-2:** Advanced map and navigation integration for route identification and ride tracking (future enhancement).
- TBD-3:** Additional localization and accessibility support (future enhancement).

Appendix D: Source Code

The following modules contain the core server-side source code of the Carpooling System and Ride Allocation application.

Module 1 — server.js

```
// — DEPENDENCIES & SETUP —————
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const path = require('path');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');
const db = require('./db');
const supabase =
  require('./supabaseClient');
const app = express();
const PORT = process.env.PORT || 3000;
const JWT_SECRET = process.env.JWT_SECRET ||
  'carpooling-secret-key-2026';
app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname,
  'public')));

// — AUTH MIDDLEWARE —————
function authenticateToken(req, res, next) {
  const authHeader =
    req.headers['authorization'];
  const token = authHeader &&
    authHeader.split(' ')[1];
  if (!token) {
    return res.status(401).json({
      success: false,
      message: 'Authentication required.'
    });
  }
  jwt.verify(token, JWT_SECRET, (err,
    userPayload) => {
    if (err)
      return res.status(403).json({
        success: false,
        message: 'Invalid/expired token.'
      });
    const user =
      db.findUserById(userPayload.userId);
    if (!user)
      return res.status(404).json({
        success: false,
        message: 'User not found.'
      });
    if (user.status === 'SUSPENDED')
      return res.status(403).json({
        success: false,
        message: 'Account suspended (FR5).'
      });
    req.user = user;
    next();
  });
}

function requireRole(...allowedRoles) {
  return (req, res, next) => {
    if (!req.user ||
      !allowedRoles.includes(req.user.role))
      return res.status(403).json({
```

```
      success: false,
      message: 'Access denied. Roles:
    ${allowedRoles.join(', ')}'
    });
    next();
  });
}

// — POST /api/auth/register —————
app.post('/api/auth/register', async (req,
  res) => {
  try {
    const {
      name, email, phone, password,
      role, licenseNo, vehicleNo,
      vehicleType, bio
    } = req.body;
    if (!name || !email || !phone ||
      !password)
      return res.status(400).json({
        success: false,
        message: 'All fields required.'
      });
    if (password.length < 6)
      return res.status(400).json({
        success: false,
        message: 'Min 6 chars (FR-2).'
      });
    if (db.findUserByEmail(email))
      return res.status(400).json({
        success: false,
        message: 'Email exists (FR-2).'
      });
    if (db.findUserByPhone(phone))
      return res.status(400).json({
        success: false,
        message: 'Phone exists (FR-2).'
      });
    const newUser = await db.createUser({
      name, email, phone, password,
      role: role || 'PASSENGER',
      licenseNo, vehicleNo,
      vehicleType, bio
    });
    await db.createNotification(
      newUser.userId, 'WELCOME',
      'Welcome!',
      `Hello ${newUser.name}, account
    created successfully.`
    );
    const token = jwt.sign(
      { userId: newUser.userId, role:
        newUser.role },
      JWT_SECRET, { expiresIn: '7d' }
    );
    const { password: _, ...userSafe } =
      newUser;
    res.status(201).json({
      success: true,
      message: 'Account registered.',
      token, user: userSafe
    });
  } catch {
    return res.status(500).json({
      success: false,
      message: 'Internal server error.'
    });
  }
});
```

```

    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: error.message
    });
  }
});

// — POST /api/auth/login —————
app.post('/api/auth/login', (req, res) => {
  try {
    const { email, password } = req.body;
    if (!email || !password)
      return res.status(400).json({
        success: false,
        message: 'Email and password
required.'
      });
    const user = db.findUserByEmail(email);
    if (!user ||
!bcrypt.compareSync(password,
user.password))
      return res.status(401).json({
        success: false,
        message: 'Invalid credentials.'
      });
    if (user.status === 'SUSPENDED')
      return res.status(403).json({
        success: false,
        message: 'Account suspended (FR5).'
      });
    const token = jwt.sign(
      { userId: user.userId, role:
user.role },
      JWT_SECRET, { expiresIn: '7d' }
    );
    const { password: , ...userSafe } =
user;
    res.json({
      success: true,
      message: 'Login successful.',
      token, user: userSafe
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: error.message
    });
  }
});

// — GET /api/auth/me —————
app.get('/api/auth/me', authenticateToken,
(req, res) => {
  const { password: , ...userSafe } =
req.user;
  res.json({ success: true, user: userSafe
});
});

// — PUT /api/auth/profile —————
app.put('/api/auth/profile',
authenticateToken, (req, res) => {
  try {
    const {
      name, phone, bio,
      licenseNo, vehicleNo, vehicleType
    } = req.body;
    const updated = db.updateUser(
      req.user.userId, {
        name, phone, bio,
        licenseNo, vehicleNo, vehicleType
      }

```

```

    );
    const { password: , ...userSafe } =
updated;
    res.json({
      success: true,
      message: 'Profile updated (FR-4).',
      user: userSafe
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: error.message
    });
  }
});

// — PUT /api/auth/change-password ———
app.put('/api/auth/change-password',
authenticateToken, (req, res) => {
  try {
    const { oldPassword, newPassword } =
req.body;
    if (!bcrypt.compareSync(oldPassword,
req.user.password))
      return res.status(400).json({
        success: false,
        message: 'Current password wrong.'
      });
    if (newPassword.length < 6)
      return res.status(400).json({
        success: false,
        message: 'Min 6 characters.'
      });
    db.updateUser(req.user.userId, {
      password: newPassword });
    res.json({
      success: true,
      message: 'Password changed.'
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: error.message
    });
  }
});

```

Module 2 — server.js

```

// — GET /api/rides — Public search —
app.get('/api/rides', (req, res) => {
  try {
    const {
      source, destination,
      date, minSeats, status
    } = req.query;
    let rides = db.getAllRides();
    // Default: ACTIVE with seats (FR-12)
    if (status)
      rides = rides.filter(r => r.status
=== status);
    else
      rides = rides.filter(r =>
        r.status === 'ACTIVE' &&
        r.availableSeats > 0);
    // Source/dest: match stops (FR-13)
    if (source) {
      const q =
source.toLowerCase().trim();
      rides = rides.filter(r =>
        r.source.toLowerCase().includes(q)
||
        r.stops.some(s =>
s.toLowerCase().includes(q)));

```

```

    }
    if (destination) {
      const q =
destination.toLowerCase().trim();
      rides = rides.filter(r =>

r.destination.toLowerCase().includes(q) ||
      r.stops.some(s =>
s.toLowerCase().includes(q)));
    }
    if (date)
      rides = rides.filter(r => r.date ===
date);
    if (minSeats)
      rides = rides.filter(r =>
      r.availableSeats >=
Number(minSeats));
    res.json({
      success: true,
      count: rides.length, rides
    });
  } catch (e) {
    res.status(500).json({
      success: false, message: e.message
    });
  }
});

// — POST /api/rides — Create (FR-6,7) —
app.post('/api/rides',
  authenticateToken,
  requireRole('DRIVER','ADMINISTRATOR'),
  (req, res) => {
    try {
      const {
        source, destination, stops, date,
        time, totalSeats, fare, notes,
        features, vehicleNo, vehicleType
      } = req.body;
      if (!source || !destination ||
!date || !time ||
        totalSeats === undefined ||
        fare === undefined)
        return res.status(400).json({
          success: false,
          message: 'All fields required
(FR6).'
        });
      if (Number(fare) < 0)
        return res.status(400).json({
          success: false,
          message: 'Fare not negative
(FR7).'
        });
      if (Number(totalSeats) <= 0 ||
        Number(totalSeats) > 10)
        return res.status(400).json({
          success: false,
          message: 'Seats: 1-10 (FR-7).'
        });
      const today = new
Date().toISOString().split('T')[0];
      if (date < today)
        return res.status(400).json({
          success: false,
          message: 'No past dates (FR-7).'
        });
      const newRide = db.createRide({
        source, destination, stops, date,
        time, totalSeats, fare, notes,
        features, vehicleNo, vehicleType
      }, req.user);
      db.createNotification(
        req.user.userId, 'RIDE_POSTED',

```

```

        'Ride Published',
        `Your ride ${newRide.source} →
${newRide.destination} is active.`
      );
      res.status(201).json({
        success: true,
        message: 'Ride created (FR-6).',
        ride: newRide
      });
    } catch (e) {
      res.status(500).json({
        success: false, message: e.message
      });
    }
  });

// — PUT /api/rides/:id — Update (FR-8) —
app.put('/api/rides/:id',
  authenticateToken, (req, res) => {
    try {
      const ride =
db.findRideById(req.params.id);
      if (!ride)
        return res.status(404).json({
          success: false,
          message: 'Ride not found.'
        });
      if (req.user.role !== 'ADMINISTRATOR'
&&
        ride.driverId !== req.user.userId)
        return res.status(403).json({
          success: false,
          message: 'Unauthorized.'
        });
      const updated =
db.updateRide(req.params.id, req.body);
      res.json({
        success: true,
        message: 'Ride updated (FR-8).',
        ride: updated
      });
    } catch (e) {
      res.status(400).json({
        success: false, message: e.message
      });
    }
  });

// — PATCH /api/rides/:id/status —
app.patch('/api/rides/:id/status',
  authenticateToken, (req, res) => {
    try {
      const { status } = req.body;
      const ride =
db.findRideById(req.params.id);
      if (!ride)
        return res.status(404).json({
          success: false,
          message: 'Ride not found.'
        });
      ride.status = status;
      db.save();
      // On COMPLETE: notify passengers
      const confirmed = db.data.bookings
        .filter(b =>
          b.rideId === ride.rideId &&
          b.status === 'CONFIRMED');
      if (status === 'COMPLETED') {
        confirmed.forEach(b => {
          b.status = 'COMPLETED';
          db.createNotification(
            b.passengerId,
            'RIDE_COMPLETED',
            'Trip Completed',

```

```

        `Rate your driver
        ${ride.driverName}!`
    );
    });
    db.save();
    } else if (status === 'CANCELLED') {
    confirmed.forEach(b => {
        b.status = 'CANCELLED';
        db.createNotification(
            b.passengerId,
            'RIDE_CANCELLED',
            'Ride Cancelled',
            `Ride ${ride.source} →
            ${ride.destination} cancelled.`
        );
    });
    db.save();
    }
    res.json({
        success: true,
        message: `Ride → ${status}`, ride
    });
    } catch (e) {
    res.status(500).json({
        success: false, message: e.message
    });
    }
    });
    });

```

Module 3 — server.js

```

// — POST /api/bookings —————
app.post('/api/bookings',
    authenticateToken,
    requireRole('PASSENGER', 'ADMINISTRATOR'),
    (req, res) => {
    try {
    const {
        rideId, seatCount,
        pickupLocation, dropLocation, notes
    } = req.body;
    if (!rideId || !seatCount)
        return res.status(400).json({
            success: false,
            message: 'Ride ID & seats
            required.'
        });
    const booking = db.createBooking({
        rideId, seatCount,
        pickupLocation, dropLocation, notes
    }, req.user);
    const ride = db.findRideById(rideId);
    // Notify driver (FR-21)
    db.createNotification(
        booking.driverId,
        'BOOKING_REQUEST',
        'New Booking Request',
        `${req.user.name} requested
        ${booking.seatCount} seat(s) on
        ${ride.source} → ${ride.destination}.`
    );
    // Notify passenger (FR-21)
    db.createNotification(
        req.user.userId,
        'BOOKING_SUBMITTED',
        'Request Submitted',
        `Your request for
        ${booking.seatCount} seat(s) is
        ${booking.status}.`
    );
    res.status(201).json({
        success: true,
        message: `Booking:
        ${booking.status} (FR-16).`,

```

```

        booking
    });
    } catch (e) {
    res.status(400).json({
        success: false, message: e.message
    });
    }
    });

// — PATCH /api/bookings/:id/status —
app.patch('/api/bookings/:id/status',
    authenticateToken, (req, res) => {
    try {
    const { status, reason } = req.body;
    const booking =
    db.findBookingById(req.params.id);
    if (!booking)
        return res.status(404).json({
            success: false,
            message: 'Booking not found.'
        });
    const isDriver = booking.driverId ===
    req.user.userId;
    const isPassenger = booking.passengerId
    === req.user.userId;
    const isAdmin = req.user.role ===
    'ADMINISTRATOR';
    if (!isDriver && !isPassenger &&
    !isAdmin)
        return res.status(403).json({
            success: false,
            message: 'Unauthorized.'
        });
    if (isPassenger && status !==
    'CANCELLED')
        return res.status(403).json({
            success: false,
            message: 'Passengers: cancel only.'
        });
    const updated = db.updateBookingStatus(
        booking.bookingId, status, reason);
    const ride =
    db.findRideById(booking.rideId);
    if (status === 'CONFIRMED')
        db.createNotification(
            booking.passengerId,
            'BOOKING_ACCEPTED',
            'Booking Confirmed!',
            `Driver approved your
            ${booking.seatCount} seat(s).`
        );
    else if (status === 'REJECTED')
        db.createNotification(
            booking.passengerId,
            'BOOKING_REJECTED',
            'Booking Declined',
            `Reason: ${reason} ||
            'Unavailable'`
        );
    else if (status === 'CANCELLED') {
    const rid = isPassenger
        ? booking.driverId
        : booking.passengerId;
    const by = isPassenger
        ? `Passenger
        ${booking.passengerName}`
        : 'Driver';
    db.createNotification(
        rid, 'BOOKING_CANCELLED',
        'Booking Cancelled',
        `${by} cancelled ride
        #${booking.rideId}.`
    );
    }

```

```

    res.json({
      success: true,
      message: `Booking → ${status} (FR-18,
FR-19).`,
      booking: updated
    });
  } catch (e) {
    res.status(400).json({
      success: false, message: e.message
    });
  }
});

```

Module 4 — server.js

```

// — POST /api/feedback —————
app.post('/api/feedback',
authenticateToken, (req, res) => {
  try {
    const {
      rideId, toUserId,
      toUserName, rating, comments
    } = req.body;
    if (!rideId || !toUserId || !rating)
      return res.status(400).json({
        success: false,
        message: 'rideId, toUserId, rating
required.'
      });
    const fb = db.createFeedback({
      rideId, toUserId, toUserName,
      rating, comments
    }, req.user);
    // Notify reviewed user (FR-21)
    db.createNotification(
      toUserId, 'NEW_FEEDBACK',
      'New Rating Received',
      `${req.user.name} rated you
${fb.rating} stars: "${fb.comments ||
'Great ride!'}"
    );
    res.status(201).json({
      success: true,
      message: 'Rating submitted (FR-25,
FR-27).',
      feedback: fb
    });
  } catch (e) {
    res.status(400).json({
      success: false, message: e.message
    });
  }
});

// — GET /api/feedback/user/:userId ———
app.get('/api/feedback/user/:userId', (req,
res) => {
  const feedbacks = db.data.feedbacks
    .filter(f => f.toUserId ===
Number(req.params.userId));
  res.json({ success: true, feedbacks });
});

```

Module 4 — db.js

```

// — db.js: createFeedback —————
createFeedback(feedbackData, fromUser) {
  const ride =
this.findRideById(feedbackData.rideId);
  if (!ride)
    throw new Error('Ride not found.');
```

// Gated to completed rides (FR-25)

```

  if (ride.status !== 'COMPLETED')

```

```

    throw new Error('Only for completed
rides (FR-25).');
    // Validate 1-5 stars (FR-26)
    const ratingVal =
Number(feedbackData.rating);
    if (isNaN(ratingVal) || ratingVal < 1 ||
ratingVal > 5)
      throw new Error('Rating must be 1-5
(FR-26).');
    const nextId = this.data.feedbacks.length
      ? Math.max(...this.data.feedbacks
        .map(f => f.feedbackId)) + 1
      : 1;
    const newFeedback = {
      feedbackId: nextId,
      rideId: ride.rideId,
      fromUserId: fromUser.userId,
      fromUserName: fromUser.name,
      fromUserRole: fromUser.role,
      toUserId:
Number(feedbackData.toUserId),
      toUserName: feedbackData.toUserName,
      rating: ratingVal,
      comments: feedbackData.comments || '',
      createdAt: new Date().toISOString()
    };
    this.data.feedbacks.unshift(newFeedback);
    // Recalculate driver avg rating (FR-28)
    const target =
this.findUserById(newFeedback.toUserId);
    if (target) {
      const all = this.data.feedbacks
        .filter(f => f.toUserId ===
target.userId);
      const total = all.reduce(
        (sum, f) => sum + f.rating, 0);
      target.totalRatingsCount = all.length;
      target.averageRating =
        Number((total /
all.length).toFixed(1));
    }
    this.save();
    return newFeedback;
  }
}

```

Module 5 — server.js

```

// — GET /api/notifications —————
app.get('/api/notifications',
authenticateToken, (req, res) => {
  const notifs =
db.getUserNotifications(req.user.userId);
  const unread = notifs.filter(n =>
!n.isRead).length;
  res.json({
    success: true,
    notifications: notifs,
    unreadCount: unread
  });
});

// — PATCH /api/notifications/:id/read —
app.patch('/api/notifications/:id/read',
authenticateToken, (req, res) => {
  const ok = db.markNotificationAsRead(
    req.params.id, req.user.userId);
  res.json({
    success: ok,
    message: ok
      ? 'Marked as read (FR-23).'
      : 'Not found.'
  });
});

```

```
// — PATCH /api/notifications/read-all —
app.patch('/api/notifications/read-all',
  authenticateToken, (req, res) => {

  db.markAllNotificationsRead(req.user.userId
);
  res.json({
    success: true,
    message: 'All read (FR-23).'
  });
});
```

Module 5 — db.js

```
// — db.js: createNotification —
createNotification(userId, type, title,
message) {
  const nextId =
this.data.notifications.length
  ? Math.max(
    ...this.data.notifications
      .map(n => n.notificationId)
  ) + 1
  : 1;
  const notif = {
    notificationId: nextId,
    userId: Number(userId),
    type, title, message,
    isRead: false,
    createdAt: new Date().toISOString()
  };
  this.data.notifications.unshift(notif);
  this.save();
  // Persist to Supabase if connected
  if (supabase) {
    supabase.from('notifications')

    .insert([notificationToSupabase(notif)])
    .then(({ error }) => {
      if (error)
        console.error(
          'Notification insert error:',
          error.message);
    });
  }
  return notif;
}

// — db.js: markAllNotificationsRead —
markAllNotificationsRead(userId) {
  this.data.notifications.forEach(n => {
    if (n.userId === Number(userId))
      n.isRead = true;
  });
  this.save();
  if (supabase) {
    supabase.from('notifications')
      .update({ is read: true })
      .eq('user_id', Number(userId));
  }
  return true;
}
```

Module 6 — server.js

```
// — GET /api/admin/stats —
app.get('/api/admin/stats',
  authenticateToken,
  requireRole('ADMINISTRATOR'),
  (req, res) => {
    const { users, rides,
      bookings, complaints } =
db.data;
    const totalEarnings = bookings
```

```
.filter(b =>
  ['COMPLETED', 'CONFIRMED'].includes(b.status
))
    .reduce((s, b) => s + b.totalFare,
0);
    res.json({ success: true, stats: {
      totalUsers: users.length,
      passengersCount: users.filter(
        u => u.role ===
'PASSENGER').length,
      driversCount: users.filter(
        u => u.role === 'DRIVER').length,
      totalRides: rides.length,
      activeRides: rides.filter(
        r => r.status === 'ACTIVE').length,
      completedRides: rides.filter(
        r => r.status ===
'COMPLETED').length,
      totalBookings: bookings.length,
      confirmedBookings: bookings.filter(
        b => b.status ===
'CONFIRMED').length,
      completedBookings: bookings.filter(
        b => b.status ===
'COMPLETED').length,
      totalFareVolume: totalEarnings,
      openComplaints: complaints.filter(
        c => ['OPEN', 'UNDER_REVIEW']
          .includes(c.status)).length
    });
  });
});
```

```
// — Admin: Suspend/Reactivate user —
app.patch('/api/admin/users/:id/status',
  authenticateToken,
  requireRole('ADMINISTRATOR'),
  (req, res) => {
    try {
      const { status } = req.body;
      if
(!['ACTIVE', 'SUSPENDED'].includes(status))
        return res.status(400).json({
          success: false,
          message: 'Invalid status.'
        });
      const user = db.updateUser(
        req.params.id, { status });
      const { password: _, ...safe } =
user;
      res.json({
        success: true,
        message: `User → ${status} (FR-
30).`,
        user: safe
      });
    } catch (e) {
      res.status(500).json({
        success: false, message: e.message
      });
    }
  });
});
```

```
// — Admin: Delete user (FR-30) —
app.delete('/api/admin/users/:id',
  authenticateToken,
  requireRole('ADMINISTRATOR'),
  (req, res) => {
    if (Number(req.params.id) ===
req.user.userId)
      return res.status(400).json({
        success: false,
        message: 'Cannot delete self.'
      });
  });
```

```

    const deleted =
db.deleteUser(req.params.id);
    res.json({
      success: deleted,
      message: deleted
        ? 'Deleted (FR-30).'
        : 'Not found.'
    });
  });

// — Admin: Database backup (FR-34) —
app.get('/api/admin/database/backup',
  authenticateToken,
  requireRole('ADMINISTRATOR'),
  (req, res) => {
    res.setHeader('Content-disposition',
      `attachment;
filename=backup_${Date.now()}.json`);
    res.setHeader('Content-type',
      'application/json');
    res.send(db.backup());
  });

// — Admin: Reset to seed data —
app.post('/api/admin/database/reset',
  authenticateToken,
  requireRole('ADMINISTRATOR'),
  (req, res) => {
    db.reset();
    res.json({
      success: true,
      message: 'Database reset (FR-34).'
    });
  });

```

Module 7 — db.js

```

// — Database class constructor & init —
class Database {
  constructor() {
    this.data = null;
    this.init();
  }

  init() {
    try {
      if (fs.existsSync(DB_FILE)) {
        const content =
fs.readFileSync(DB_FILE, 'utf8');
        this.data = JSON.parse(content);
      } else {
        this.data = getInitialData();
        this.save();
      }
    } catch (e) {
      this.data = getInitialData();
      this.save();
    }
  }

  // Sync with Supabase on startup —
  async initSupabaseSync() {
    if (!supabase) return;
    const [uR, rR, bR, fR, nR, cR] =
      await Promise.all([
        supabase.from('users')
          .select('*')
          .order('user_id', { ascending:
true }),
        supabase.from('rides')
          .select('*')
          .order('ride_id', { ascending:
true }),

```

```

        supabase.from('bookings')
          .select('*')
          .order('booking id', { ascending:
true }),
        supabase.from('feedbacks')
          .select('*'),
        supabase.from('notifications')
          .select('*'),
        supabase.from('complaints')
          .select('*')
      ]);
    // Upload missing local users
    const emails = new Set(
      (uR.data || []).
        .map(u => u.email.toLowerCase()));
    const missing = this.data.users
      .filter(u =>
!emails.has(u.email.toLowerCase()));
    for (const u of missing)
      await supabase.from('users')
        .upsert([userToSupabase(u)]);
    // Replace local with Supabase data
    if (uR.data?.length)
      this.data.users =
uR.data.map(userFromSupabase);
    if (rR.data?.length)
      this.data.rides =
rR.data.map(rideFromSupabase);
    if (bR.data?.length)
      this.data.bookings =
bR.data.map(bookingFromSupabase);
    if (fR.data?.length)
      this.data.feedbacks =
fR.data.map(feedbackFromSupabase);
    if (nR.data?.length)
      this.data.notifications =
nR.data.map(notificationFromSupabase);
    if (cR.data?.length)
      this.data.complaints =
cR.data.map(complaintFromSupabase);
    this.save();
  }

  save() {
    fs.writeFileSync(DB_FILE,
      JSON.stringify(this.data, null, 2),
      'utf8');
  }

  backup() {
    return JSON.stringify(this.data, null,
2);
  }

  reset() {
    this.data = getInitialData();
    this.save();
  }

  // — Booking with atomic seat lock —
  createBooking(bookingData, passenger) {
    const ride =
this.findRideById(bookingData.rideId);
    if (!ride || ride.status !== 'ACTIVE')
      throw new Error('Ride unavailable.');
```

```

    // Conflict detection (FR-20)
    const conflict = this.data.bookings
      .find(b =>
        b.passengerId === passenger.userId
    &&
        b.rideId === ride.rideId &&
        ['CONFIRMED', 'PENDING'].includes(b.status))
    ;
    if (conflict)
      throw new Error('Duplicate booking
exists (FR-20).');
    const nextId =
this.data.bookings.length
  ? Math.max(...this.data.bookings
    .map(b => b.bookingId)) + 1
    : 1001;
    const autoApprove =
this.data.systemSettings.autoApproveBooking
s;
    const newBooking = {
      bookingId: nextId,
      rideId: ride.rideId,
      passengerId: passenger.userId,
      passengerName: passenger.name,
      passengerPhone: passenger.phone,
      driverId: ride.driverId,
      driverName: ride.driverName,
      seatCount: seats,
      totalFare: ride.fare * seats,
      status: autoApprove ? 'CONFIRMED' :
'PENDING',
      bookingDate: new
Date().toISOString(),
      pickupLocation:
        bookingData.pickupLocation ||
ride.source,
      dropLocation:
        bookingData.dropLocation ||
ride.destination,
      notes: bookingData.notes || ''
    };
    // Deduct seats if auto-approve
    if (autoApprove)
      ride.availableSeats -= seats;
    this.data.bookings.unshift(newBooking);
    this.save();
    // Persist to Supabase...
    return newBooking;
  }

  // — Update status with seat reconcile —
  updateBookingStatus(bookingId, newStatus,
reason = '') {
    const booking =
this.findBookingById(bookingId);
    const ride =
this.findRideById(booking.rideId);
    // CONFIRMED: deduct seats
    if (newStatus === 'CONFIRMED') {
      if (ride.availableSeats <
booking.seatCount)
        throw new Error('Not enough seats
remaining.');
```

```

      ride.availableSeats -=
booking.seatCount;
    }
    // CANCEL/REJECT: restore seats
    if
    ([ 'CANCELLED', 'REJECTED' ].includes(newStatu
s) &&
      booking.status === 'CONFIRMED')
      ride.availableSeats = Math.min(
        ride.totalSeats,
        ride.availableSeats +
        booking.seatCount);
    booking.status = newStatus;
    if (reason) booking.statusReason =
reason;
    this.save();
    return booking;
  }
}
```

Module 8 — supabaseClient.js

```

require('dotenv').config();
const { createClient } =
require('@supabase/supabase-js');

const supabaseUrl =
process.env.SUPABASE_URL;
const supabaseKey =
process.env.SUPABASE_KEY;

let supabase = null;

if (supabaseUrl && supabaseKey &&
supabaseUrl.trim() !== '' &&
supabaseKey.trim() !== '') {
  try {
    supabase = createClient(
      supabaseUrl.trim(),
      supabaseKey.trim()
    );
    console.log(
      '[Supabase] Connected to remote
Supabase instance:', supabaseUrl
    );
  } catch (err) {
    console.error(
      '[Supabase] Initialization error:',
err.message
    );
  }
} else {
  console.log(
    '[Database] Supabase credentials not
found in .env. Running with local
persistent data engine.'
  );
  console.log(
    'To connect: Add SUPABASE_URL and
SUPABASE_KEY into .env and run
supabase_schema.sql'
  );
}

module.exports = supabase;
```

Appendix E: Demonstration

The following screenshots demonstrate the working Carpooling System and Ride Allocation application.

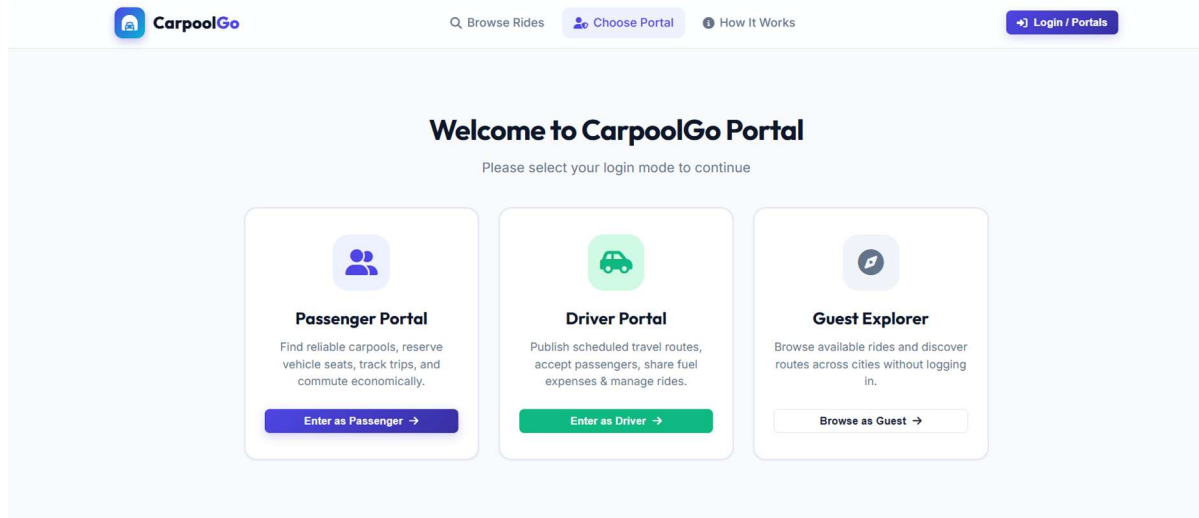


Figure 12: Portal Selection (Login Page)

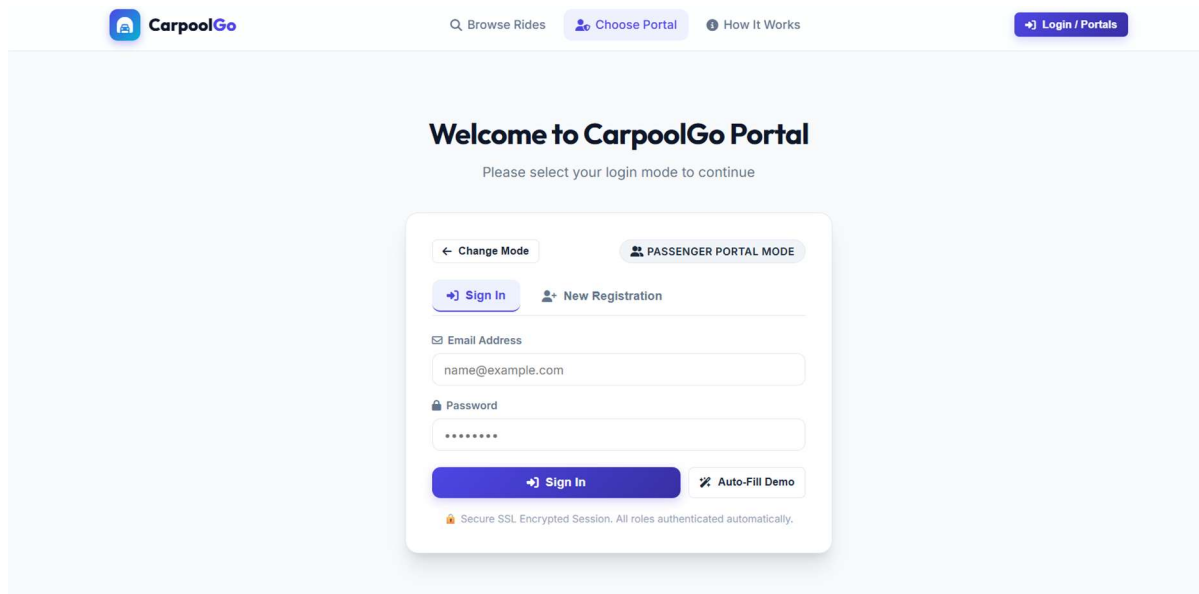
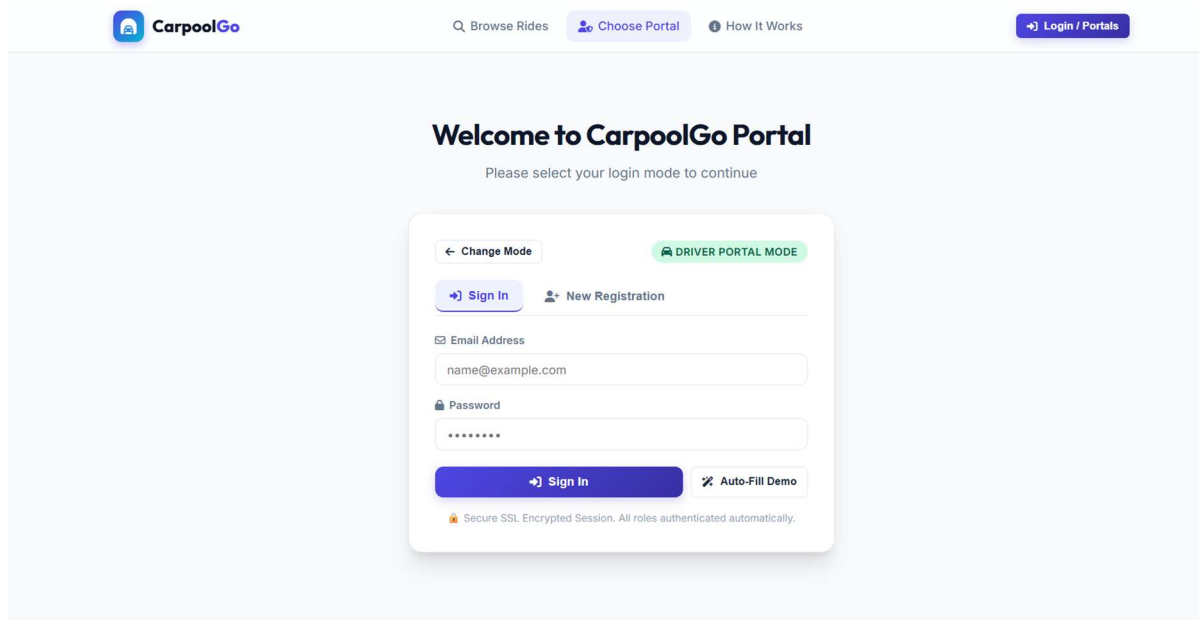
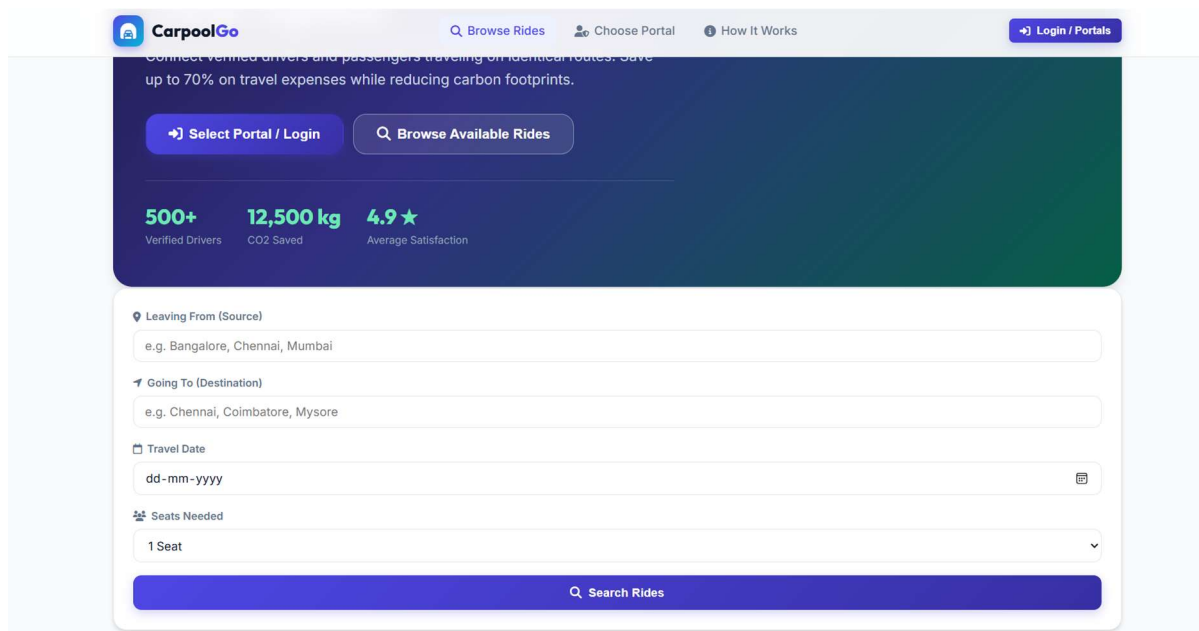


Figure 13: Passenger Login



The screenshot shows the 'Welcome to CarpoolGo Portal' page. At the top, there is a navigation bar with the CarpoolGo logo, links for 'Browse Rides', 'Choose Portal', 'How It Works', and a 'Login / Portals' button. The main heading is 'Welcome to CarpoolGo Portal' with a subtext 'Please select your login mode to continue'. Below this is a login form with a 'Change Mode' button and a 'DRIVER PORTAL MODE' indicator. The form includes 'Sign In' and 'New Registration' buttons, an 'Email Address' field with the placeholder 'name@example.com', and a 'Password' field with masked characters. There is a 'Sign In' button at the bottom of the form and an 'Auto-Fill Demo' button. A security notice at the bottom states: 'Secure SSL Encrypted Session. All roles authenticated automatically.'

Figure 14: Driver Login



The screenshot shows the 'Guest Login' page. At the top, there is a navigation bar with the CarpoolGo logo, links for 'Browse Rides', 'Choose Portal', 'How It Works', and a 'Login / Portals' button. Below the navigation bar is a dark blue banner with the text: 'Connect verified drivers and passengers traveling on identical routes. Save up to 70% on travel expenses while reducing carbon footprints.' Below the banner are two buttons: 'Select Portal / Login' and 'Browse Available Rides'. Below these buttons are three statistics: '500+ Verified Drivers', '12,500 kg CO2 Saved', and '4.9 ★ Average Satisfaction'. Below the statistics is a form with four fields: 'Leaving From (Source)' with the placeholder 'e.g. Bangalore, Chennai, Mumbai', 'Going To (Destination)' with the placeholder 'e.g. Chennai, Coimbatore, Mysore', 'Travel Date' with the placeholder 'dd-mm-yyyy' and a calendar icon, and 'Seats Needed' with a dropdown menu showing '1 Seat'. At the bottom of the form is a 'Search Rides' button.

Figure 15: Guest Login

The screenshot displays the CarpoolGo app's search interface. At the top, there's a navigation bar with the CarpoolGo logo, a search icon, and links for 'Browse Rides' and 'My Bookings'. A user profile for 'kishor v PASSENGER' is visible in the top right. The main search area contains several input fields: 'Leaving From (Source)' with 'chennai', 'Going To (Destination)' with 'coimbatore', 'Travel Date' with '03-10-2026', and 'Seats Needed' with '1 Seat'. A blue 'Search Rides' button is at the bottom of this section. Below the search bar, a message states 'Found 1 Matching Ride(s)' with a 'Refresh List' link. The ride details for 'Arun Sundaram' (4.8 rating, SUV Hyundai Creta) are shown, including the route 'Chennai (Anna Nagar) to Coimbatore (Gandhipuram)' via Villupuram, Salem, and Erode. The fare is ₹750 per seat. Additional details include the date '2026-10-03', time '05:00', and '4 of 5 seats open'. Amenities like AC, Spacious, Music Allowed, and Pets Allowed are listed. A license plate 'TN-09-CB-4490' is also shown. A note at the bottom mentions 'Early morning start. Comfortable SUV, coffee stops on bypass.'

Figure 16: Ride Searching

The screenshot shows the 'Book Ride Seats' modal in the CarpoolGo app. The modal title is 'Book Ride Seats'. It displays the selected ride details: 'Chennai (Anna Nagar) → Coimbatore (Gandhipuram)', 'Date: 2026-10-03 | Departure: 05:00 hrs', and 'Driver: Arun Sundaram (SUV (Hyundai Creta) • Reg: TN-09-CB-4490)'. Below this, there's a section for 'Number of Seats to Reserve' with a dropdown set to '1 Seat (₹750)'. The 'Pickup Location / Landmark' is 'Chennai (Anna Nagar)' and the 'Drop-off Location / Destination' is 'Coimbatore (Gandhipuram)'. There's a field for 'Special Instructions / Luggage details (Optional)' with the text 'e.g. 1 small backpack, will reach 5 mins early'. At the bottom, the 'Total Estimated Fare' is ₹750. Two buttons are at the bottom: 'Cancel' and 'Submit Booking Request'.

Figure 17: Ride Booking – Book Ride Seats

Offer a Ride

Source City / Area: chennai

Destination City / Area: coimbatore

Intermediate Waypoint Stops (Comma separated): hosur

Date of Journey: 29-09-2026

Departure Time: --:--

Seats Offered (1 - 8): 3

Fare Per Seat (₹): 100

Vehicle Reg Number: TN07BB0724

Vehicle Model / Type: honda city

Trip Amenities / Features (Comma separated): AC, Music, Fastag, Non-Smoking

Ride Rules & Guidelines for Passengers: no pets allowed

Buttons: Cancel, Publish Ride Offer

Figure 18: Ride Posting – Driver Offer a Ride

Driver Dashboard
Manage your published ride offers, approve passenger requests, and track earnings

Summary: ₹0 Total Completed Earnings, 2 Active Listed Trips, 4.9★ Driver Rating Score, 0 Trips Completed

My Posted Rides & Allocations | Passenger Requests

Published Ride Offers

Ride ID	Route & Stops	Schedule	Seats (Avail/Total)	Fare	Status	Actions
#RD-101	Bangalore (Electronic City) – Chennai (Guindy) Via: Hosur, Krishnagiri, Vellore, Sriperumbudur	2026-10-02 06:30 hrs	3 / 4 Available	₹650	ACTIVE	Complete Cancel
#RD-103	Bangalore (Koramangala) – Mysore (Suburban Bus Stand) Via: Kengeri, Bidadi, Mandya	2026-09-29 17:30 hrs	1 / 3 Available	₹350	ACTIVE	Complete Cancel

Figure 19: Driver Ride List

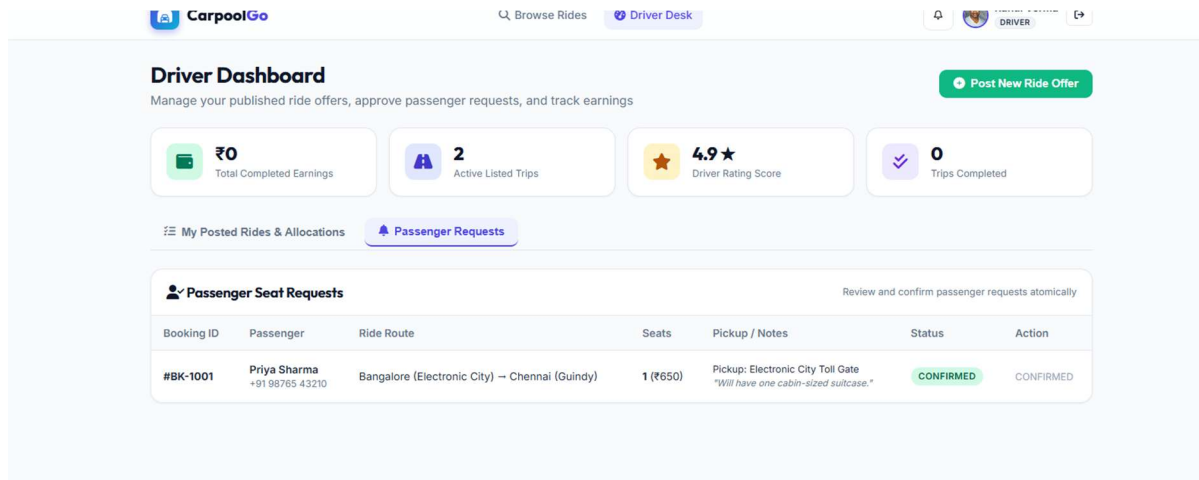


Figure 20: Driver – Passenger Requests

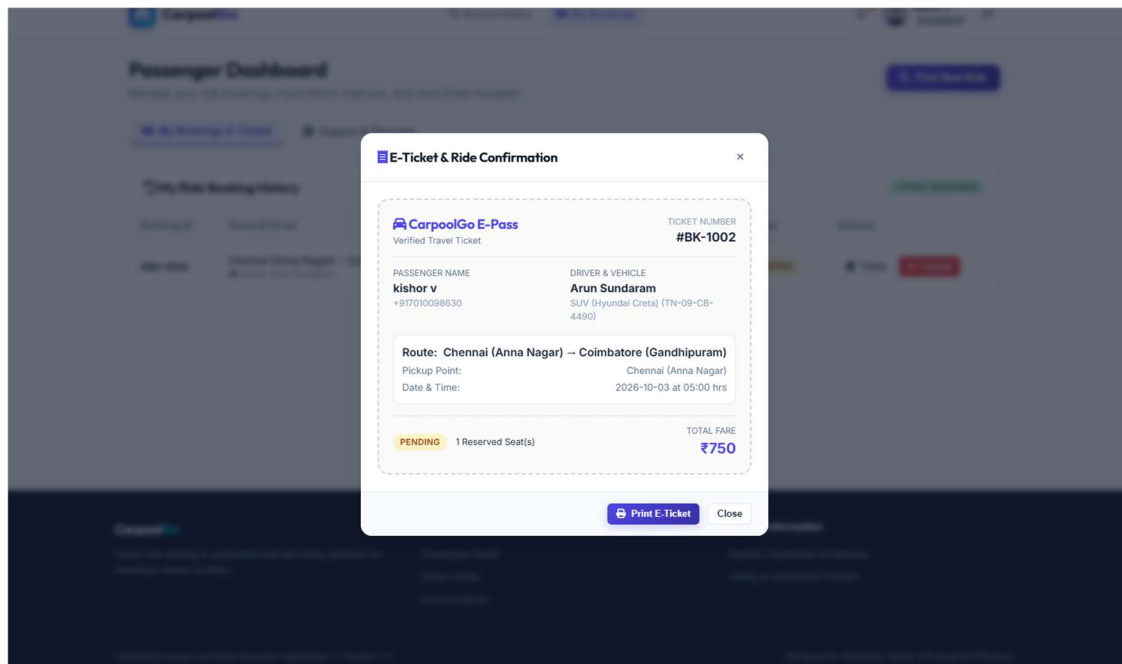


Figure 21: E-Ticket and Ride Confirmation

Appendix F: E-Resource Link

GitHub Repository:

<https://github.com/Kishorkumar146/carpooling-system.git>